EXP:6	
21/8/2026	**CDN Service Selection and Configuration**

AIM

To select a Content Delivery Network (CDN) provider, deploy a simple web application using Cloudflare, access the application through Cloudflare's edge network, and verify the HTTP response and Cloudflare-specific headers using browser Developer Tools.

THEORY

Content Delivery Network (CDN)

A Content Delivery Network (CDN) is a distributed network of servers or edge locations used to deliver web content closer to users. Instead of every request having to travel directly to a distant origin server, CDN/edge infrastructure can process and deliver content from locations distributed across the network. This can reduce latency and improve the user experience.

Why CDN is required?

A CDN reduces the distance that content needs to travel by serving requests through distributed edge locations. This can improve response time and the user experience.

Cloudflare

Cloudflare provides services such as CDN, DNS, security, and edge computing. In this experiment, Cloudflare Workers were used to deploy a simple application on Cloudflare's distributed edge network and demonstrate how a user's request can be processed through the cloud edge.

CDN PROVIDER SELECTION

Provider	Purpose
Cloudflare	CDN + security + DNS + edge computing
Amazon CloudFront	AWS CDN
Akamai	Enterprise CDN

Cloudflare was selected because it was the easiest option for the practical experiment.

REQUIREMENTS

- Cloudflare account
- Web browser (Google Chrome used for the experiment)
- Internet connection
- Cloudflare Workers dashboard

PROCEDURE

Step 1 – Select the CDN/Edge Provider – Cloudflare was selected as the CDN/edge provider. The Cloudflare account was opened and no DNS or nameserver settings were changed.

Step 2 – Open the Create App screen – From the Cloudflare dashboard, the Create app workflow was opened.

Step 3 – Select Start with Hello World! – The Start with Hello World! option was selected because GitHub or GitLab was not required for this experiment.

Step 4 – Keep the generated Worker name and default code – The generated Worker name was retained. The default Worker code returning Hello World! was left unchanged for the first deployment.

Step 5 – Deploy the Worker – The Deploy button was clicked. Cloudflare created the Worker and provided a public workers.dev address.

Step 6 – Test the Worker – The Visit button was clicked. The browser displayed Hello World!, confirming that the Worker was accessible.

Step 7 – Create the CDN demonstration page – The Worker code was edited and replaced with an HTML page explaining Cloudflare content delivery and CDN benefits.

Step 8 – Deploy the updated page – The modified Worker was deployed again and accessed using the Visit button.

Step 9 – Verify the CDN response – Chrome Developer Tools were opened using F12. Network was selected and the page was reloaded using Ctrl + R. The workers.dev request was selected and its Headers were inspected.

PROGRAM / WORKER CODE

```
export default {
  async fetch(request, env, ctx) {
    return new Response(`
      <!DOCTYPE html>
      <html>
      <head>
        <title>CDN Experiment</title>
      </head>
      <body>
        <h1>CDN Service Experiment</h1>
        <h2>Cloudflare Content Delivery</h2>
        <p>This webpage is being delivered through Cloudflare.</p>
        <p>CDN reduces the distance between users and content.</p>
        <p>Status: Successfully Deployed</p>
      </body>
      </html>
    `, {
      headers: {
        "content-type": "text/html"
      }
    });
```

```
    }
};
```

OUTPUT / OBSERVATIONS

The following screenshots were captured during the experiment to document deployment, application output, and HTTP header verification.

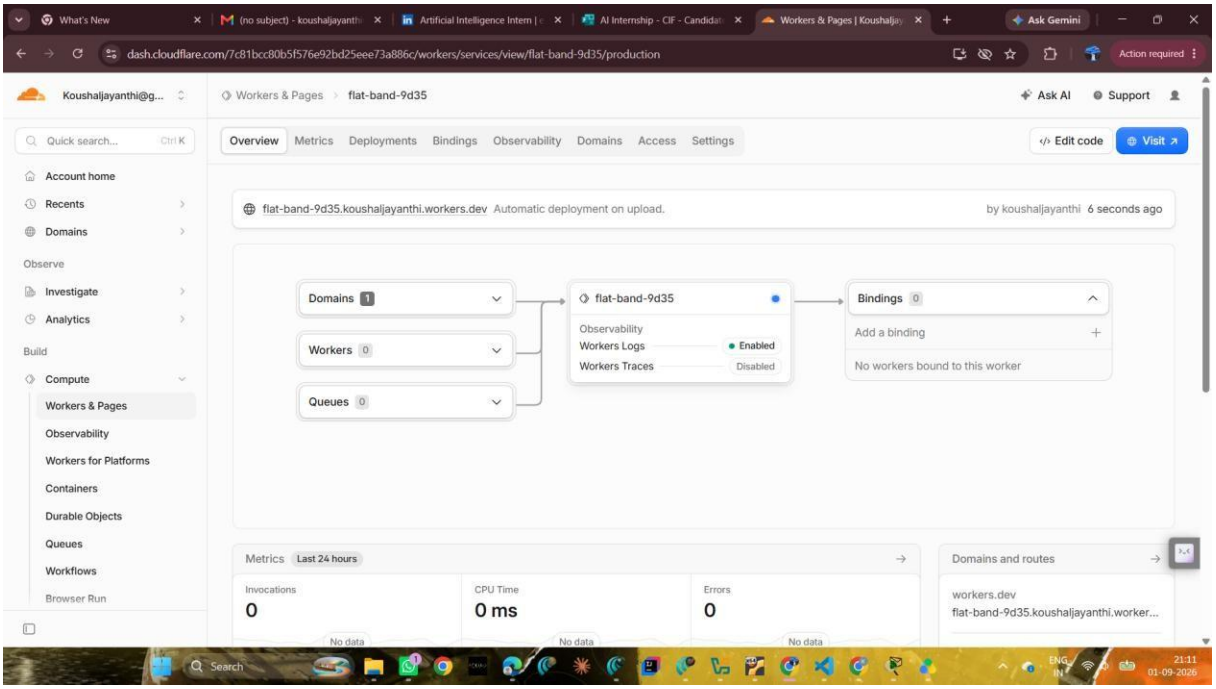


Figure 1: Cloudflare Worker successfully deployed with a workers.dev domain.

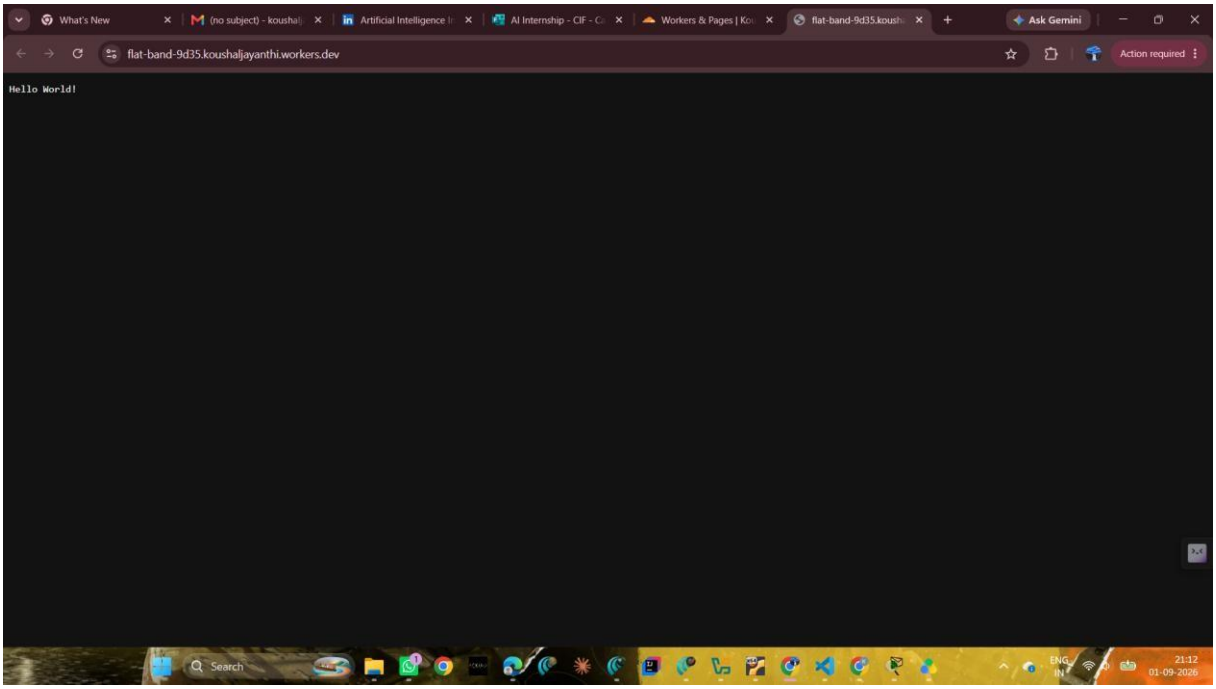


Figure 2: Initial Worker test showing the Hello World! response.

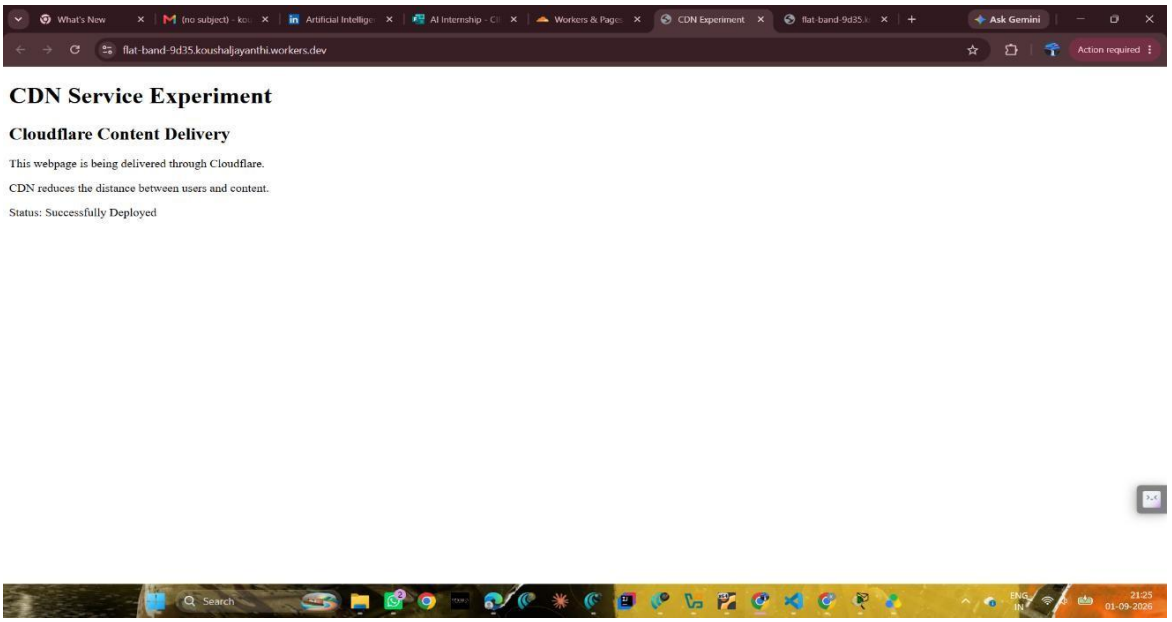


Figure 3: Final CDN Service Experiment webpage deployed through Cloudflare.

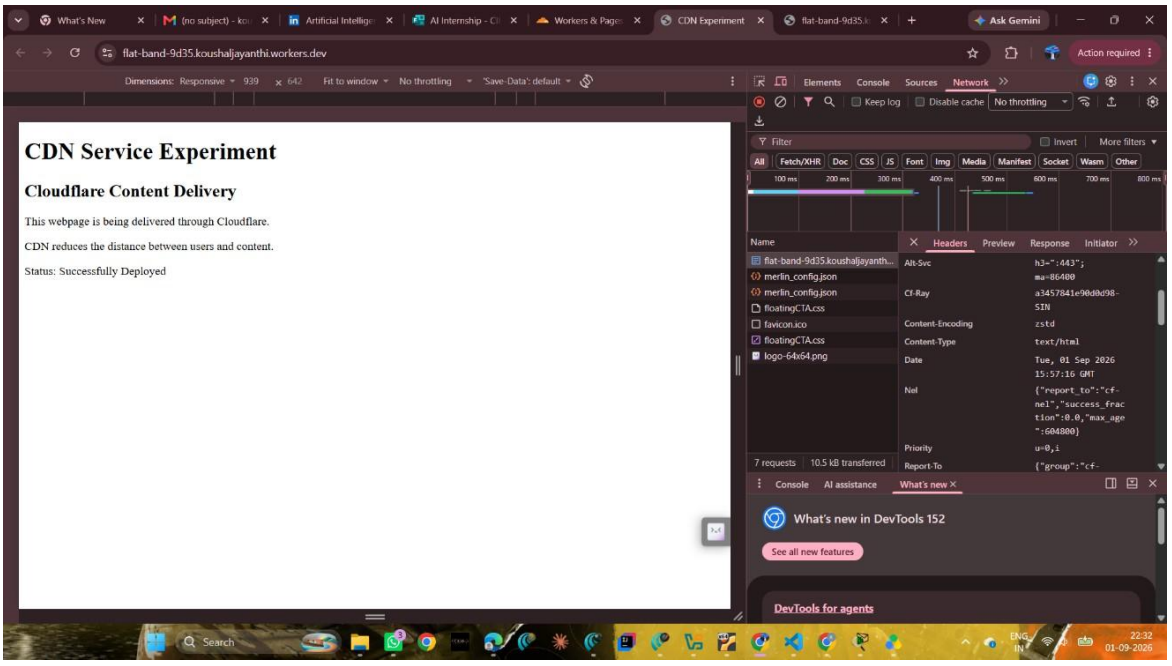


Figure 4: Response headers showing CF-Ray and Content-Type: text/html.

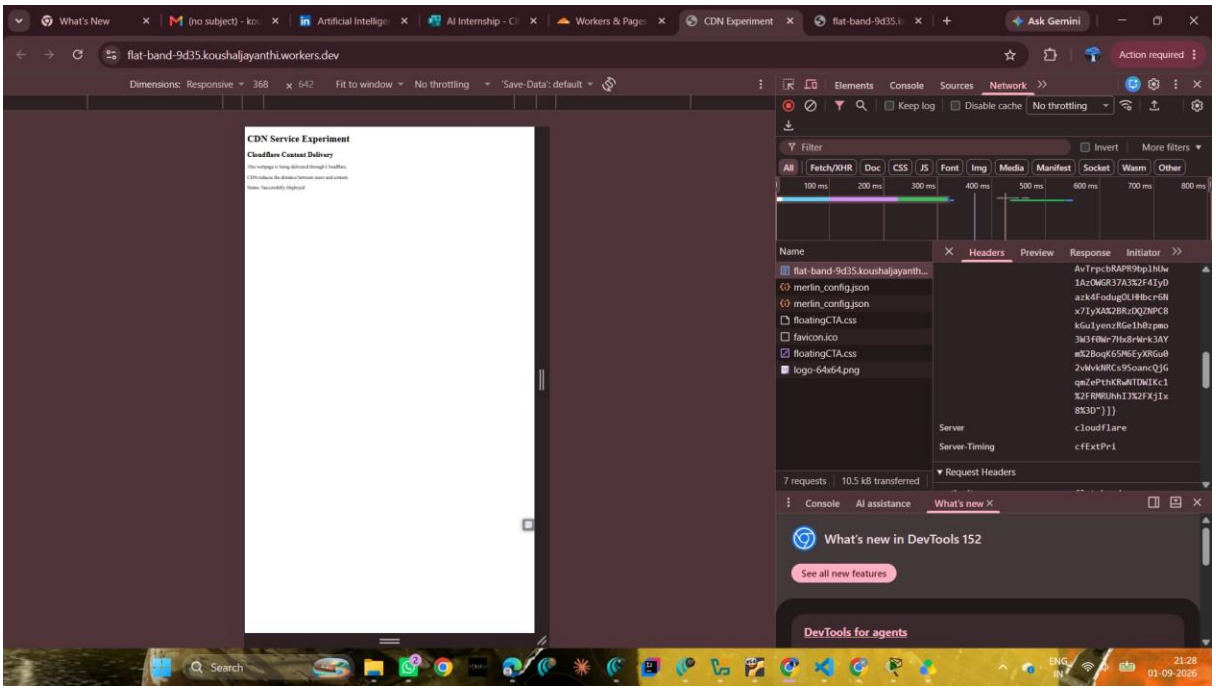


Figure 5: Response headers showing Server: cloudflare.

HTTP HEADER VERIFICATION

Field	Observed Value	Inference
Request Method	GET	The browser requested the webpage.
Status Code	200 OK	The request was successfully received and the webpage was returned.
Server	cloudflare	The response was handled through Cloudflare.
CF-Ray	a3457841e90d0d98-…-SIN	Cloudflare request identifier present in the response.
Content-Type	text/html	The response contains an HTML webpage.

CONCLUSION

Thus, CDN service selection and configuration were successfully demonstrated using Cloudflare Workers. The deployed webpage was accessible through the Cloudflare edge network and the response was verified using browser Developer Tools.